Pack 25 (Web + API) — Device-Admin Audit Trail: Who Trusted / Blocked / Granted (real code)

RemoteDesk · Pack 25 (Web + API) — Device-Admin Audit Trail: Who Trusted / Blocked / Granted (real code)

> Codex / Claude Code handoff. Direct continuation of Packs 23 + 24. Pack 24 locked down WHO can change trust / mint grants, but nothing records WHEN they did or WHAT changed. This slice wires every device-admin mutation (trust set, grant create, grant revoke, receipt verify-fail) into the Pack 14 audit log, with before/after diffs, and adds a per-device admin history view in the web dashboard. Build order: shared audit-action contract → audit helper → route hooks → web history view → tests.
> Scope: an append-only, non-sensitive record of every privileged device-admin action, attributable to the acting user, reusing the Pack 14 AuditLog. **Content-free diffs (status/scope/expiry only, no secrets). Audit write never blocks the mutation it records but failures are surfaced. Reuses existing Pack 14 + Pack 24 machinery, no new persistence model.**
> Tag legend: **[NEW]** = create file. **[MODIFY]** = patch existing file by hand (merge, don't overwrite).

Files created / modified / not touched

```
CREATED [NEW]
  packages/shared/src/permissions/deviceAdminAudit.ts
  apps/api/src/lib/deviceAdminAudit.ts
  apps/web/app/dashboard/devices/DeviceAuditHistory.tsx
  tests/admin/deviceAdminAudit.test.ts
  tests/admin/deviceAdminAudit.lib.test.ts

MODIFIED [MODIFY]
  packages/shared/src/permissions/index.ts                  (export deviceAdminAudit)
  packages/shared/src/audit/types.ts                         (add device-admin AuditActions)
  packages/shared/src/audit/schema.ts                        (extend the action enum)
  apps/api/src/services/admin/deviceTrust.service.ts         (audit trust set w/ before→after)
  apps/api/src/services/admin/unattendedGrant.service.ts     (audit grant create + revoke)
  apps/api/src/services/admin/receiptVerifier.service.ts     (audit receipt verify failures)
  apps/api/src/routes/deviceAdmin.routes.ts                  (pass actor + expose /:deviceId/audit)
  apps/web/app/dashboard/devices/[deviceId]/page.tsx          (render DeviceAuditHistory)
  apps/web/app/dashboard/devices/deviceAdminApi.ts            (deviceAudit fetch)

NOT TOUCHED (intentionally)
  apps/desktop/**                        (no desktop change; admin actions are web/API)
  apps/api/prisma/**                     (reuses Pack 14 AuditLog; no new model)
```

Part A — Shared: device-admin audit contract

FILE: packages/shared/src/permissions/deviceAdminAudit.ts

```
import type { DeviceTrustStatus } from './devicePolicy';
import type { FeatureKey } from './devicePolicy';

/**
 * Content-free descriptors for device-admin audit events. These build the
 * `metadata` map for a Pack 14 AuditLog row, so they carry ONLY non-sensitive
 * fields (status names, scope feature keys, ISO expiry, ids) — never secrets,
 * fingerprintsignals, or payloads.
 */

/** Trust change: before -> after. */
export function trustChangeMetadata(input: {
  deviceId: string;
  from: DeviceTrustStatus | null;
  to: DeviceTrustStatus;
}): Record<string, string> {
  return {
    deviceId: input.deviceId,
    from: input.from ?? 'unknown',
    to: input.to,
  };
}

/** Grant creation: device + clamped scope + window. */
export function grantCreateMetadata(input: {
  deviceId: string;
  grantId: string;
  scope: FeatureKey[];
  notBefore: string;
  expiresAt: string;
}): Record<string, string> {
  return {
    deviceId: input.deviceId,
    grantId: input.grantId,
    scope: input.scope.join(','),
    notBefore: input.notBefore,
    expiresAt: input.expiresAt,
  };
}

/** Grant revocation. */
export function grantRevokeMetadata(input: { deviceId: string; grantId: string }): Record<string, string> {
  return { deviceId: input.deviceId, grantId: input.grantId };
}

/** Receipt verification failure (at store or at rest). */
export function receiptVerifyFailMetadata(input: {
  receiptId: string;
  reason: string;
  firstBrokenSeq: number | null;
}): Record<string, string> {
  return {
    receiptId: input.receiptId,
    reason: input.reason,
    firstBrokenSeq: input.firstBrokenSeq === null ? 'none' : String(input.firstBrokenSeq),
  };
```

```
}

/** Human label for a device-admin audit action (UI display). */
export function deviceAdminActionLabel(action: string): string {
  switch (action) {
    case 'device.trust.changed': return 'Trust changed';
    case 'device.grant.created': return 'Unattended grant created';
    case 'device.grant.revoked': return 'Unattended grant revoked';
    case 'device.receipt.verify_failed': return 'Receipt verification failed';
    default: return action;
  }
}
```

FILE: packages/shared/src/audit/types.ts [MODIFY]

```
// ---- MODIFY: extend the AuditAction union with device-admin actions ----
// export type AuditAction =
//   | ...existing Pack 14 actions...
//   | 'device.trust.changed'
//   | 'device.grant.created'
//   | 'device.grant.revoked'
//   | 'device.receipt.verify_failed';
```

FILE: packages/shared/src/audit/schema.ts [MODIFY]

```
// ---- MODIFY: add the four new actions to auditActionSchema enum ----
// export const auditActionSchema = z.enum([
//   ...existing...,
//   'device.trust.changed',
//   'device.grant.created',
//   'device.grant.revoked',
//   'device.receipt.verify_failed',
// ]);
```

FILE: packages/shared/src/permissions/index.ts [MODIFY]

```
// ...existing exports through Pack 24...
export * from './deviceAdminAudit'; // ADD
```

Part B — API: device-admin audit helper

FILE: apps/api/src/lib/deviceAdminAudit.ts

```
import type { AuditAction } from '@remotedesk/shared';
import { auditLogger } from './auditLogger'; // Pack 14

/**
 * Thin wrapper that records a device-admin action into the Pack 14 audit log,
 * attributed to the acting user + device. Reuses auditLogger (which already
 * redacts metadata + asserts content-free). A failure here is logged but does
 * NOT throw, so it never rolls back the privileged mutation that already happened.
 */
export const deviceAdminAudit = {
  async record(input: {
```

```
    orgId: string;
    actorUserId: string | null;
    deviceId: string;
    action: AuditAction;
    metadata: Record<string, string>;
  }): Promise<void> {
    try {
      await auditLogger.record({
        orgId: input.orgId,
        action: input.action,
        userId: input.actorUserId,
        deviceId: input.deviceId,
        metadata: input.metadata,
      });
    } catch (err) {
      // Non-fatal: the mutation succeeded; surface for ops without breaking the request.
      console.warn('[device-admin-audit] failed to record', {
        action: input.action,
        deviceId: input.deviceId,
      });
    }
  },
};
```

Part C — API: service hooks

FILE: apps/api/src/services/admin/deviceTrust.service.ts [MODIFY]

```
// ...existing Pack 23 imports...
import { trustChangeMetadata } from '@remotedesk/shared';
import { deviceAdminAudit } from '../../lib/deviceAdminAudit';

// ---- MODIFY setTrust: capture before -> after + audit ----
// async setTrust(orgId, deviceId, status, actorUserId, fingerprint?) {   // ADD actorUserId param
//   const policy = await deviceTrustRepository.policy(deviceId, orgId);
//   if (!policy) throw new AppError(ErrorCodes.DEVICE_NOT_FOUND);
//   const from = policy.trustStatus;
//   await deviceTrustRepository.setTrust(deviceId, orgId, status, fingerprint);
//   await deviceAdminAudit.record({
//     orgId, actorUserId, deviceId,
//     action: 'device.trust.changed',
//     metadata: trustChangeMetadata({ deviceId, from, to: status }),
//   });
// }
```

FILE: apps/api/src/services/admin/unattendedGrant.service.ts [MODIFY]

```
// ...existing Pack 23 imports...
import { grantCreateMetadata, grantRevokeMetadata } from '@remotedesk/shared';
import { deviceAdminAudit } from '../../lib/deviceAdminAudit';

// ---- MODIFY create: audit after the (clamped) grant is created ----
// const grant = await unattendedGrantRepository.create({ ... });
// await deviceAdminAudit.record({
//   orgId, actorUserId: createdById, deviceId,
```

```
//   action: 'device.grant.created',
//   metadata: grantCreateMetadata({
//     deviceId, grantId: grant.grantId, scope: grant.scope,
//     notBefore: grant.notBefore, expiresAt: grant.expiresAt,
//   }),
// });
// return grant;

// ---- MODIFY revoke: accept actor + deviceId, audit ----
// async revoke(grantId, orgId, actorUserId, deviceId) {
//   await unattendedGrantRepository.revoke(grantId, orgId);
//   await deviceAdminAudit.record({
//     orgId, actorUserId, deviceId,
//     action: 'device.grant.revoked',
//     metadata: grantRevokeMetadata({ deviceId, grantId }),
//   });
// }
```

FILE: apps/api/src/services/admin/receiptVerifier.service.ts [MODIFY]

```
// ...existing Pack 24 imports...
import { receiptVerifyFailMetadata } from '@remotedesk/shared';
import { deviceAdminAudit } from '../../lib/deviceAdminAudit';

// ---- MODIFY storeVerified: audit a rejection before throwing ----
// const result = await validateReceipt(receipt, nodeSha256Hex);
// if (!result.valid) {
//   await deviceAdminAudit.record({
//     orgId, actorUserId: null, deviceId: receipt.summary.deviceId,
//     action: 'device.receipt.verify_failed',
//     metadata: receiptVerifyFailMetadata({
//       receiptId: receipt.receiptId, reason: result.reason ?? 'unknown', firstBrokenSeq: result.firstBrokenSeq,
//     }),
//   });
//   throw new AppError(ErrorCodes.RECEIPT_INVALID);
// }
```

Part D — API: routes

FILE: apps/api/src/routes/deviceAdmin.routes.ts [MODIFY]

```
// ...existing Pack 23/24 imports...
import { auditRepository } from '../repositories/auditRepository'; // Pack 14

// ---- MODIFY mutation routes: thread the acting userId + deviceId into services ----
// .patch('/:deviceId/trust', requireAuth, orgContext, requireDeviceAdmin, h(async (req, res) => {
//   const { trustStatus } = setTrustSchema.parse(req.body);
//   const r = req as OrgRequest & AuthedRequest;
//   await deviceTrustService.setTrust(r.orgId, req.params.deviceId, trustStatus, r.userId);
//   res.status(204).end();
// }))
// .post('/:deviceId/grants', requireAuth, orgContext, requireDeviceAdmin, h(async (req, res) => {
//   const input = createGrantSchema.parse(req.body);
//   const r = req as OrgRequest & AuthedRequest;
//   res.status(201).json(await unattendedGrantService.create(r.orgId, req.params.deviceId, input as any, r.userId));
```

```
// }))
// .delete('/:deviceId/grants/:grantId', requireAuth, orgContext, requireDeviceAdmin, h(async (req, res) => {
//   const r = req as OrgRequest & AuthedRequest;
//   await unattendedGrantService.revoke(req.params.grantId, r.orgId, r.userId, req.params.deviceId);
//   res.status(204).end();
// }))

// ---- ADD: per-device admin audit history (read; any org member) ----
// .get('/:deviceId/audit', requireAuth, orgContext, h(async (req, res) => {
//   const logs = await auditRepository.list((req as OrgRequest).orgId, 200);
//   // Filter to device-admin actions for THIS device.
//   const deviceActions = new Set(['device.trust.changed', 'device.grant.created', 'device.grant.revoked',
'device.receipt.verify_failed']);
//   res.json(logs.filter((l) => deviceActions.has(l.action) && l.actor.deviceId === req.params.deviceId));
// }))
```

Part E — Web: per-device audit history

FILE: apps/web/app/dashboard/devices/deviceAdminApi.ts [MODIFY]

```
// ...existing Pack 23 imports...
import type { AuditLogDto } from '@remotedesk/shared';

// ---- ADD to deviceAdminApi: ----
// deviceAudit: (deviceId: string) => api.get<AuditLogDto[]>(`/devices/${deviceId}/audit`),
```

FILE: apps/web/app/dashboard/devices/DeviceAuditHistory.tsx

```
'use client';

import React, { useEffect, useState } from 'react';
import type { AuditLogDto } from '@remotedesk/shared';
import { deviceAdminActionLabel } from '@remotedesk/shared';
import { deviceAdminApi } from './deviceAdminApi';
import styles from './devices.module.css';

/**
 * Per-device admin history: who trusted/blocked the device, who minted/revoked
 * grants, and any receipt verification failures. Read-only; content-free metadata.
 */
export function DeviceAuditHistory({ deviceId }: { deviceId: string }) {
  const [logs, setLogs] = useState<AuditLogDto[]>([]);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    deviceAdminApi.deviceAudit(deviceId).then(setLogs).catch(() => setError('Could not load history.'));
  }, [deviceId]);

  return (
    <section className={styles.panel}>
      <h2>Admin history</h2>
      {error && <div className={styles.error}>{error}</div>}
      {logs.length === 0 ? (
        <p className={styles.muted}>No admin actions recorded for this device.</p>
      ) : (
```

```
        <table className={styles.table}>
          <thead><tr><th>When</th><th>Action</th><th>By</th><th>Detail</th></tr></thead>
          <tbody>
            {logs.map((l) => (
              <tr key={l.id}>
                <td>{new Date(l.createdAt).toLocaleString()}</td>
                <td>{deviceAdminActionLabel(l.action)}</td>
                <td>{l.actor.userId ?? 'system'}</td>
                <td className={styles.mono}>{summarize(l.metadata)}</td>
              </tr>
            ))}
          </tbody>
        </table>
      )}
    </section>
  );
}

/** Render the content-free metadata compactly. */
function summarize(metadata: Record<string, string>): string {
  if (metadata.from && metadata.to) return `${metadata.from} → ${metadata.to}`;
  if (metadata.scope) return `scope: ${metadata.scope} · expires ${metadata.expiresAt ?? '?'}`;
  if (metadata.grantId && !metadata.scope) return `grant ${metadata.grantId.slice(0, 8)}…`;
  if (metadata.reason) return `${metadata.reason}${metadata.firstBrokenSeq !== 'none' ? ` @${metadata.firstBrokenSeq}` :
''}`;
  return Object.entries(metadata).map(([k, v]) => `${k}=${v}`).join(' · ');
}
```

FILE: apps/web/app/dashboard/devices/[deviceId]/page.tsx [MODIFY]

```
// ...existing Pack 23/24 imports...
import { DeviceAuditHistory } from '../DeviceAuditHistory';

// ---- ADD at the bottom of the device detail render (visible to any member) ----
// <DeviceAuditHistory deviceId={deviceId} />
//
// Existing trust panel + grants panel (admin-gated from Pack 24) stay as-is.
```

Part F — Tests

FILE: tests/admin/deviceAdminAudit.test.ts

```
import { describe, it, expect } from 'vitest';
import {
  trustChangeMetadata,
  grantCreateMetadata,
  grantRevokeMetadata,
  receiptVerifyFailMetadata,
  deviceAdminActionLabel,
} from '@remotedesk/shared';

describe('deviceAdminAudit metadata', () => {
  it('builds a before->after trust change', () => {
    expect(trustChangeMetadata({ deviceId: 'd1', from: 'untrusted', to: 'trusted' }))
      .toEqual({ deviceId: 'd1', from: 'untrusted', to: 'trusted' });
```

```
  });

  it('represents an unknown previous status', () => {
    expect(trustChangeMetadata({ deviceId: 'd1', from: null, to: 'blocked' }).from).toBe('unknown');
  });

  it('serializes grant scope as a comma list', () => {
    const m = grantCreateMetadata({
      deviceId: 'd1', grantId: 'g1', scope: ['remoteInput', 'clipboardSync'] as any,
      notBefore: '2026-06-15T00:00:00.000Z', expiresAt: '2026-06-16T00:00:00.000Z',
    });
    expect(m.scope).toBe('remoteInput,clipboardSync');
    expect(m.grantId).toBe('g1');
  });

  it('builds revoke + verify-fail metadata', () => {
    expect(grantRevokeMetadata({ deviceId: 'd1', grantId: 'g1' })).toEqual({ deviceId: 'd1', grantId: 'g1' });
    const vf = receiptVerifyFailMetadata({ receiptId: 'r1', reason: 'chain_invalid', firstBrokenSeq: 3 });
    expect(vf.reason).toBe('chain_invalid');
    expect(vf.firstBrokenSeq).toBe('3');
    expect(receiptVerifyFailMetadata({ receiptId: 'r1', reason: 'bad_version', firstBrokenSeq: null
}).firstBrokenSeq).toBe('none');
  });

  it('labels actions for display', () => {
    expect(deviceAdminActionLabel('device.trust.changed')).toBe('Trust changed');
    expect(deviceAdminActionLabel('device.grant.revoked')).toBe('Unattended grant revoked');
    expect(deviceAdminActionLabel('unknown.thing')).toBe('unknown.thing');
  });

  it('metadata is content-free (no secret-looking values)', () => {
    const m = grantCreateMetadata({ deviceId: 'd1', grantId: 'g1', scope: ['remoteInput'] as any, notBefore: 'x',
expiresAt: 'y' });
    const json = JSON.stringify(m);
    expect(json).not.toMatch(/token|password|secret|authorization|cookie/i);
  });
});
```

FILE: tests/admin/deviceAdminAudit.lib.test.ts

```
import { describe, it, expect, vi, beforeEach } from 'vitest';

const record = vi.fn();
vi.mock('../../apps/api/src/lib/auditLogger', () => ({
  auditLogger: { record: (...a: unknown[]) => record(...a) },
}));

import { deviceAdminAudit } from '../../apps/api/src/lib/deviceAdminAudit';

describe('deviceAdminAudit.record (lib)', () => {
  beforeEach(() => record.mockReset());

  it('forwards to the Pack 14 auditLogger with actor + device', async () => {
    record.mockResolvedValue(undefined);
    await deviceAdminAudit.record({
      orgId: 'org', actorUserId: 'u1', deviceId: 'd1',
```

```
      action: 'device.trust.changed', metadata: { from: 'untrusted', to: 'trusted' },
    });
    expect(record).toHaveBeenCalledWith({
      orgId: 'org', action: 'device.trust.changed', userId: 'u1', deviceId: 'd1',
      metadata: { from: 'untrusted', to: 'trusted' },
    });
  });

  it('never throws when the audit write fails (non-fatal)', async () => {
    record.mockRejectedValue(new Error('db down'));
    await expect(deviceAdminAudit.record({
      orgId: 'org', actorUserId: 'u1', deviceId: 'd1',
      action: 'device.grant.revoked', metadata: { grantId: 'g1' },
    })).resolves.toBeUndefined();
  });
});
```

Part G — Reports

FILE: generated-pack25-web-api-device-admin-audit-merge-summary.md

```
# Pack 25 (Web + API) — Device-Admin Audit Trail · Merge Summary

## What this slice does
Continues Packs 23 + 24. Records WHO did WHAT and WHEN for every privileged
device-admin action, reusing the Pack 14 audit log (no new model):
- device.trust.changed (before -> after)
- device.grant.created (clamped scope + window)
- device.grant.revoked
- device.receipt.verify_failed (reason + first broken seq)
Plus a per-device "Admin history" view in the web dashboard.

## Files created (5)
- packages/shared/src/permissions/deviceAdminAudit.ts
- apps/api/src/lib/deviceAdminAudit.ts
- apps/web/app/dashboard/devices/DeviceAuditHistory.tsx
- tests/admin/deviceAdminAudit.test.ts
- tests/admin/deviceAdminAudit.lib.test.ts

## Files modified (9)
- packages/shared/src/permissions/index.ts — export deviceAdminAudit.
- packages/shared/src/audit/types.ts + schema.ts — add 4 device-admin AuditActions.
- apps/api/src/services/admin/deviceTrust.service.ts — audit trust set (before->after) + actorUserId param.
- apps/api/src/services/admin/unattendedGrant.service.ts — audit grant create + revoke (actor + deviceId).
- apps/api/src/services/admin/receiptVerifier.service.ts — audit verify failures before throwing.
- apps/api/src/routes/deviceAdmin.routes.ts — thread actor/deviceId; add GET /:deviceId/audit.
- apps/web/app/dashboard/devices/[deviceId]/page.tsx — render DeviceAuditHistory.
- apps/web/app/dashboard/devices/deviceAdminApi.ts — deviceAudit fetch.

## Files intentionally NOT touched
- apps/desktop/** — admin actions are web/API only.
- apps/api/prisma/** — reuses Pack 14 AuditLog; no migration.

## Remaining manual wiring risks
1. setTrust + grant revoke service signatures gain an actorUserId (and deviceId for revoke); update all callers.
```

```
2. Add the 4 actions to the AuditAction union + zod enum (or POST /audit/logs will reject them).
3. The /:deviceId/audit route filters AuditLog by actor.deviceId; ensure auditLogger stores deviceId (it does in Pack 14).
4. Confirm Pack 14 auditLogger redaction won't strip the content-free metadata keys used here (it won't; they're non-sensitive).

## Verify
- pnpm --filter @remotedesk/shared build && pnpm -r test
- Manual: trust a device -> Admin history shows "Trust changed: untrusted -> trusted" by your user; create + revoke a grant -> two rows; POST a tampered receipt -> a verify_failed row appears.
```

FILE: generated-pack25-web-api-device-admin-audit-code-review.md

```
# Pack 25 (Web + API) — Code Review Notes

## Strengths
- Reuses the Pack 14 audit log + redaction; no new persistence model, no schema change.
- Audit metadata is content-free by construction (status/scope/expiry/ids only); a test asserts no secret-looking values.
- Audit write is non-fatal: a logging failure never rolls back the privileged mutation it records.
- Trust change captures before->after by reading policy prior to the update.
- Per-device history is read-open to members; mutations remain admin-gated (Pack 24).

## Reviewer checklist
- [ ] All callers of setTrust / grant revoke updated for the new actor params.
- [ ] AuditAction union + zod enum include the 4 new actions.
- [ ] auditLogger persists deviceId (the history route filters on it).
- [ ] Receipt verify-fail audit fires before the thrown 400.
- [ ] No fingerprint signals / payloads leak into metadata (verified by shape).

## Safety
- Every privileged device-admin action is attributable + timestamped.
- Content-free: the audit trail cannot leak secrets or session content.
- Append-only via Pack 14; host still owns live permissions.
```

FILE: generated-pack25-web-api-device-admin-audit-manifest.json


```
{
  "pack": "pack25-web-api-device-admin-audit",
  "title": "Device-Admin Audit Trail: Who Trusted / Blocked / Granted",
  "actualFileCount": 14,
  "filesCreated": [
    "packages/shared/src/permissions/deviceAdminAudit.ts",
    "apps/api/src/lib/deviceAdminAudit.ts",
    "apps/web/app/dashboard/devices/DeviceAuditHistory.tsx",
    "tests/admin/deviceAdminAudit.test.ts",
    "tests/admin/deviceAdminAudit.lib.test.ts"
  ],
  "filesModified": [
    "packages/shared/src/permissions/index.ts",
    "packages/shared/src/audit/types.ts",
    "packages/shared/src/audit/schema.ts",
    "apps/api/src/services/admin/deviceTrust.service.ts",
    "apps/api/src/services/admin/unattendedGrant.service.ts",
    "apps/api/src/services/admin/receiptVerifier.service.ts",
    "apps/api/src/routes/deviceAdmin.routes.ts",
    "apps/web/app/dashboard/devices/[deviceId]/page.tsx",
```

```
      "apps/web/app/dashboard/devices/deviceAdminApi.ts"
    ],
    "filesNotTouched": [
      "apps/desktop/**",
      "apps/api/prisma/**"
    ],
    "dependenciesRequired": {
      "new": [],
      "existing": ["@remotedesk/shared", "express", "next", "react", "vitest"],
      "assumedFromProject": [
        "Pack 14 auditLogger + auditRepository + AuditLog model (stores deviceId)",
        "Pack 23 device-admin services + routes",
        "Pack 24 receipt verifier + RBAC",
        "orgContext providing orgId + actor userId"
      ]
    },
    "knownLimitations": [
      "History route filters the org audit log by deviceId in-app; add a dedicated indexed query if volume is high.",
      "Receipt verify failures are attributed to 'system' (no user actor on the desktop POST path).",
      "Audit trail proves WHO/WHEN/WHAT-changed, not cryptographic non-repudiation (a later signing step).",
      "No new model; relies on Pack 14 retention settings."
    ],
    "remainingManualWiringRisks": [
      "Update setTrust / grant-revoke callers for the new actor params.",
      "Add the 4 actions to the AuditAction union + zod enum.",
      "Confirm auditLogger stores deviceId for the history filter."
    ],
    "estimatedCompletionAfterCarefulMerge": "80-85%"
  }
```

Pack 25 (Web + API) done: 14 files (5 created, 9 modified). Every privileged device-admin action now leaves an attributable, timestamped trail in the Pack 14 audit log: who trusted or blocked a device (with before→after), who minted or revoked an unattended grant (with the clamped scope + window), and any receipt that failed verification (with the reason + first broken sequence). The web device-detail page gains an "Admin history" panel showing it all.

The clean bit: zero new persistence, it rides the existing Pack 14 audit log + redaction, so there's one audit pipeline, not two. The metadata is content-free by construction (status names, scope keys, ISO expiry, ids, nothing else), and a test asserts no secret-looking values can appear. Audit writes are non-fatal: if logging fails, the privileged mutation it records still stands rather than rolling back on a logging hiccup.

That gives the trust/consent system full accountability: locked down in 24, now fully attributable in 25. Next natural slice = cross-device receipt signing (Ed25519) for real non-repudiation beyond internal consistency, or a SIEM/webhook export of the device-admin audit stream. Bolo.